Covacha MCP Server - Servidor MCP Propio para el Ecosistema

Fecha: 2026-03-21 **Autor:** Cesar / Claude Code **Estado:** Diseno de arquitectura

---

1. Por Que Un MCP Server Propio

Context Mode resuelve el problema **generico** de ahorro de contexto. Pero nuestro ecosistema tiene necesidades especificas que ningun MCP server publico cubre:

Necesidad	Context Mode	Covacha MCP
Ahorro de contexto generico	Si	Hereda de Context Mode
Coordinacion 4 maquinas	No	**Si — DynamoDB nativo**
GitHub Board awareness	No	**Si — sync + mutations**
Contratos de API	No	**Si — registry nativo**
Dependency resolution	No	**Si — DEP# queries**
Auto-evaluacion de tickets	No	**Si — quality gates**
Roles de equipo (tester, PO)	No	**Si — role-based tools**
Learnings compartidos	No	**Si — LEARNING# queries**
Multi-tenant context	No	**Si — tenant-aware**

Estrategia: Context Mode como **base** (ahorro de tokens + session continuity) + Covacha MCP como **capa de negocio** (coordinacion + roles + evaluacion).

---

2. Arquitectura del MCP Server

```
┌──────────────────────────────────────────────────────────────────┐
│                     Claude Code Session                          |
|                                                                 |
|  MCP Servers:                                                   |
|  ┌──────────────────┐  ┌──────────────────────────────────┐   |
|  |  Context Mode     |  |  Covacha MCP Server              |   |
|  |  (ahorro tokens)  |  |  (coordinacion + negocio)        |   |
|  |                   |  |                                   |   |
|  |  ctx_execute      |  |  covacha_claim_task              |   |
|  |  ctx_batch_execute|  |  covacha_release_task            |   |
|  |  ctx_index        |  |  covacha_team_status             |   |
|  |  ctx_search       |  |  covacha_send_message            |   |
|  |  ctx_fetch_index  |  |  covacha_read_messages           |   |
|  |  ctx_execute_file |  |  covacha_publish_contract        |   |
|  |                   |  |  covacha_get_contract            |   |
|  |  (session hooks)  |  |  covacha_check_dependencies      |   |
|  └──────────────────┘  |  covacha_evaluate_ticket         |   |
|                        |  covacha_bootstrap                |   |
```

```
│                          │  covacha_heartbeat                  │  │
│                          │  covacha_get_learnings              │  │
│                          │  covacha_save_learning              │  │
│                          │  covacha_dispatcher_suggest         │  │
│                          │  covacha_role_action                │  │
│                          └─────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────────┘
          │                          │
          ▼                          ▼
   SQLite local              DynamoDB remoto
   (session events)          (covacha-agent-memory)
                                     │
                                     ▼
                            GitHub Project Board
                            (fuente de verdad)
```

2.1 Stack tecnico

Componente	Tecnologia	Razon
MCP Server	TypeScript (Node.js)	Estandar MCP, compatible con Claude Code
Transport	stdio	Estandar para MCP servers locales
Backend	Python scripts existentes	Reutiliza agent_memory/ completo
Database	DynamoDB	Ya existe, single-table
Local cache	SQLite	Para cache de queries frecuentes

2.2 Estructura del proyecto

```
covacha-mcp-server/
├── package.json
├── tsconfig.json
├── src/
│   ├── index.ts              # Entry point MCP server
│   ├── server.ts             # MCP server setup + tool registration
│   ├── tools/
│   │   ├── task-tools.ts     # claim, release, bootstrap, find_next
│   │   ├── message-tools.ts  # send, read, mark_as_read
│   │   ├── contract-tools.ts # publish, get, search
│   │   ├── dep-tools.ts      # add, check, resolve dependencies
│   │   ├── team-tools.ts     # status, heartbeat, stale check
│   │   ├── eval-tools.ts     # evaluate ticket, quality gates
│   │   ├── role-tools.ts     # role-based actions (tester, PO, dev)
│   │   └── learning-tools.ts # get, save learnings
│   ├── clients/
│   │   ├── dynamo.ts         # DynamoDB client (AWS SDK v3)
│   │   ├── github.ts         # GitHub GraphQL + REST
│   │   └── cache.ts          # SQLite local cache
│   ├── roles/
│   │   ├── developer.ts      # Acciones de desarrollador
│   │   ├── tester.ts         # Acciones de tester/QA
│   │   ├── project-owner.ts  # Acciones de PO
│   │   ├── reviewer.ts       # Acciones de code reviewer
```

```
│   │   └── types.ts          # Role interfaces
│   └── config.ts             # Constantes (mismas que config.py)
├── tests/
│   └── ...
└── README.md
```

---

3. Herramientas MCP (Tools)

3.1 Task Management

`covacha_bootstrap`

Genera contexto fresco para la sesion actual.

```
{
  name: "covacha_bootstrap",
  description: "Genera CONTEXT.md con tareas disponibles, learnings, es
  inputSchema: {
    type: "object",
    properties: {
      team: { type: "string", enum: ["backend", "frontend", "testing",
      machine: { type: "string", enum: ["mac-1", "mac-2", "mac-3", "mac
      module: { type: "string", description: "Repo/modulo para cargar l
    },
    required: ["team", "machine"]
  }
}
```

Retorna: CONTEXT.md con tareas, learnings, estado del equipo, modelo recomendado.

`covacha_claim_task`

Reclama una tarea con lock atomico.

```
{
  name: "covacha_claim_task",
  inputSchema: {
    properties: {
      task_id: { type: "string" },
      team: { type: "string" },
      machine: { type: "string" }
    },
    required: ["task_id", "team", "machine"]
  }
}
```

Retorna: Exito/fallo + branch name + modelo recomendado + contratos relevantes.

`covacha_release_task`

Libera una tarea y guarda learnings.

```
{
  name: "covacha_release_task",
  inputSchema: {
    properties: {
      task_id: { type: "string" },
      team: { type: "string" },
      machine: { type: "string" },
      status: { type: "string", enum: ["done", "blocked", "cancelled"]
      learnings: { type: "array", items: { type: "string" } },
      contract: {
        type: "object",
        properties: {
          name: { type: "string" },
          method: { type: "string" },
          path: { type: "string" },
          request_schema: { type: "object" },
          response_schema: { type: "object" }
        }
      }
    },
    required: ["task_id", "team", "machine", "status"]
  }
}
```

Retorna: Confirmacion + issue cerrado + board actualizado + dependientes desbloqueados.

3.2 Communication

`covacha_send_message`

```
{
  name: "covacha_send_message",
  inputSchema: {
    properties: {
      type: { type: "string", enum: ["task_completed", "task_blocked",
      task_id: { type: "string" },
      payload: { type: "object" },
      to_machine: { type: "string", description: "Opcional: mensaje dir
    },
    required: ["type"]
  }
}
```

`covacha_read_messages`

```
{
  name: "covacha_read_messages",
  inputSchema: {
    properties: {
      machine: { type: "string" },
      type_filter: { type: "string", description: "Filtrar por tipo de
      mark_as_read: { type: "boolean", default: false }
```

```
    },
    required: ["machine"]
  }
}
```

3.3 Dependencies

covacha_check_dependencies

```
{
  name: "covacha_check_dependencies",
  inputSchema: {
    properties: {
      task_id: { type: "string" },
      action: { type: "string", enum: ["check", "add", "resolve", "list
    },
    required: ["task_id", "action"]
  }
}
```

Retorna: Estado de dependencias, tareas bloqueadas/desbloqueadas.

3.4 Contracts

covacha_publish_contract

```
{
  name: "covacha_publish_contract",
  inputSchema: {
    properties: {
      name: { type: "string" },
      version: { type: "string" },
      method: { type: "string" },
      path: { type: "string" },
      request_schema: { type: "object" },
      response_schema: { type: "object" },
      task_id: { type: "string" }
    },
    required: ["name", "version", "method", "path"]
  }
}
```

covacha_get_contract

```
{
  name: "covacha_get_contract",
  inputSchema: {
    properties: {
      contract_id: { type: "string" },
      search_path: { type: "string", description: "Buscar por path patt
    }
  }
```

```
}
```

3.5 Auto-Evaluacion de Tickets

`covacha_evaluate_ticket`

Esta es la herramienta clave. Evalua si un ticket esta realmente completo.

```
{
  name: "covacha_evaluate_ticket",
  inputSchema: {
    properties: {
      task_id: { type: "string" },
      evaluation_type: { type: "string", enum: ["pre_release", "post_me
    },
    required: ["task_id", "evaluation_type"]
  }
}
```

Evaluaciones por tipo:

`pre_release` (antes de hacer release_task)

```
Checklist automatico:
[ ] Tests escritos y pasando (pytest -v exit code 0)
[ ] Coverage >= 98% en archivos modificados
[ ] Ruff check sin errores
[ ] Ningun archivo > 1000 lineas
[ ] Nunguna funcion > 50 lineas
[ ] Type hints en todas las funciones nuevas
[ ] Docstrings en funciones publicas
[ ] Commit format correcto: tipo(ISS-XXX): descripcion
[ ] Branch pushed al remote
[ ] No hay secrets hardcodeados (.env, tokens)
```

`post_merge` (despues de merge a main)

```
Checklist automatico:
[ ] CI/CD pipeline exitoso
[ ] Deploy completado sin errores
[ ] No hay rollbacks en los ultimos 30 min
[ ] Metricas de Grafana estables
[ ] No hay errores nuevos en logs
```

`qa_review` (mac-3 evalua trabajo de otra maquina)

```
Checklist:
[ ] Tests cubren happy path
[ ] Tests cubren al menos 1 caso de error
[ ] Tests cubren edge cases (vacios, nulos, limites)
[ ] No hay tests que dependan de orden de ejecucion
[ ] Mocks correctos para servicios externos
[ ] No hay datos hardcodeados en tests
```

`po_acceptance` (evaluacion de criterios de aceptacion)

```
Checklist:
[ ] Cada criterio de aceptacion de la US tiene un test
[ ] El comportamiento cumple con la descripcion del issue
[ ] No hay regresiones en funcionalidad existente
[ ] UX cumple (si es frontend)
[ ] API contract publicado (si es backend)
```

---

4. Sistema de Roles

4.1 Roles por maquina

Cada maquina no solo tiene una especializacion tecnica, sino un **ROL** en el equipo:

Maquina	Rol tecnico	Rol de equipo	Responsabilidades
mac-1	Backend Core	**Tech Lead**	Arquitectura, decisiones tecnicas, contracts de API, code review backend
mac-2	Frontend	**Developer**	Implementacion UI, integracion con contracts, UX
mac-3	Testing + QA	**Tester / QA**	Quality gates, evaluacion de tickets, regression tests, E2E
mac-4	Ops + Integ	**Project Owner**	Priorizacion, aceptacion de tickets, tracking de epicas, coordinacion

4.2 Tool: `covacha_role_action`

```
{
  name: "covacha_role_action",
  inputSchema: {
    properties: {
      role: { type: "string", enum: ["tech_lead", "developer", "tester"
      action: { type: "string" },
      task_id: { type: "string" },
      payload: { type: "object" }
    },
    required: ["role", "action"]
  }
}
```

4.3 Acciones por Rol

Tech Lead (mac-1)

Accion	Que hace
`review_architecture`	Evalua si el diseno de una tarea es correcto antes de implementar
`publish_contract`	Publica contrato de API para que frontend lo consuma
`approve_pr`	Revisa y aprueba PR de otra maquina (code review)

`define_dependencies`	Define que tareas dependen de cuales
`escalate_blocker`	Escala un blocker enviando MSG a todas las maquinas

Developer (mac-2)

Accion	Que hace
`implement_from_contract`	Lee contrato publicado y genera codigo frontend
`request_contract`	Pide a mac-1 que publique un contrato que necesita
`report_integration_issue`	Reporta que el contrato no funciona como esperado
`update_component_docs`	Documenta componentes creados

Tester / QA (mac-3)

Accion	Que hace
`evaluate_ticket`	Ejecuta quality gates completos sobre una tarea
`write_regression_tests`	Escribe tests de regresion para features existentes
`run_e2e_suite`	Ejecuta suite E2E completa y reporta resultados
`review_test_coverage`	Evalua coverage de un PR y sugiere tests faltantes
`report_bug`	Crea issue de bug en GitHub + MSG a la maquina responsable

Project Owner (mac-4)

Accion	Que hace
`accept_ticket`	Evalua criterios de aceptacion y cierra/rechaza
`prioritize_backlog`	Reordena prioridades en el board
`update_epic_status`	Actualiza estado de epica en covacha-projects
`generate_sprint_report`	Genera reporte de progreso del sprint
`create_cross_repo_issue`	Crea issue CROSS que toca multiples repos
`sync_board`	Fuerza sync GitHub Board → DynamoDB

4.4 Flujo de evaluacion de ticket (auto-evaluacion)

```
┌──────────────────────────────────────────────────────────────┐
│            FLUJO DE EVALUACION DE TICKET                      │
│                                                               │
│  mac-1 (developer): Completa ISS-043                          │
│    │                                                          │
│    ▼                                                          │
│  covacha_evaluate_ticket(043, "pre_release")                  │
│    → Ejecuta checklist automatico                             │
│    → Tests pasan? Coverage >= 98%? Lint limpio?               │
│    │                                                          │
│    ├─ PASS → release_task(043, done)                          │
│    │         → MSG: "ISS-043 ready for QA"                    │
│    │                                                          │
```

```
│    └─ FAIL → Fix issues → Re-evaluate                    │
│                                                           │
│  mac-3 (tester): Recibe MSG "ready for QA"                │
│    │                                                      │
│    ▼                                                      │
│  covacha_evaluate_ticket(043, "qa_review")                │
│    → Evalua tests, coverage, edge cases                   │
│    │                                                      │
│    ├─ PASS → MSG: "ISS-043 QA approved"                   │
│    │                                                      │
│    └─ FAIL → report_bug() → MSG a mac-1 con detalles      │
│                                                           │
│  mac-4 (PO): Recibe MSG "QA approved"                     │
│    │                                                      │
│    ▼                                                      │
│  covacha_evaluate_ticket(043, "po_acceptance")            │
│    → Evalua criterios de aceptacion de la US              │
│    │                                                      │
│    ├─ PASS → accept_ticket(043)                           │
│    │         → Cierra issue, mueve board a Done           │
│    │         → Actualiza epica en covacha-projects        │
│    │                                                      │
│    └─ FAIL → MSG a mac-1 con criterios no cumplidos       │
│                                                           │
│  mac-4 (PO): Si toda la epica esta completa               │
│    → update_epic_status(EP-SP-019, "COMPLETADO")          │
│    → Actualiza .md en covacha-projects                    │
│    → generate_sprint_report()                             │
└───────────────────────────────────────────────────────────┘
```

---

5. Comunicacion Inter-Maquina Detallada

5.1 Tipos de mensaje y flujos

```
MSG Types:
──────────────────────────────────────────────────────────────
task_completed      mac-1 → broadcast   "Termine ISS-043"
task_blocked        mac-2 → broadcast   "ISS-044 bloqueada por X"
contract_published  mac-1 → broadcast   "Nuevo contract: SPEI Transfer
review_requested    mac-1 → mac-3       "Review mi PR #87"
qa_approved         mac-3 → mac-4       "ISS-043 paso QA"
qa_rejected         mac-3 → mac-1       "ISS-043 fallo QA: falta test X
po_accepted         mac-4 → broadcast   "ISS-043 aceptada, epica actual
po_rejected         mac-4 → mac-1       "ISS-043 no cumple criterio Y"
help_needed         any   → broadcast   "Necesito ayuda con Z"
bug_found           mac-3 → mac-1       "Bug en endpoint /api/v1/..."
dependency_resolved mac-1 → mac-2       "ISS-043 done, puedes empezar 0
sprint_report       mac-4 → broadcast   "Reporte: 12/15 tareas completa
```

5.2 Protocolo de comunicacion

```
1. BROADCAST: Mensaje visible para todas las maquinas
   - TTL: 24 horas
   - read_by: [] → se llena conforme cada maquina lee
   - Uso: task_completed, contract_published, sprint_report

2. DIRECT: Mensaje para una maquina especifica
   - TTL: 24 horas
   - to_machine: "mac-3"
   - Uso: review_requested, qa_rejected, bug_found

3. URGENT: Mensaje que requiere atencion inmediata
   - TTL: 1 hora
   - priority: "high"
   - Uso: help_needed, blocker escalation
   - Maquina receptora lo procesa en siguiente heartbeat (2 min max)
```

5.3 Message Processing por rol

```
def process_message(msg, team, machine, role):
    """Cada rol procesa mensajes de forma diferente."""

    if role == "tester":
        if msg["type"] == "task_completed":
            # Automaticamente evalua el ticket
            evaluate_ticket(msg["task_id"], "qa_review")

        elif msg["type"] == "review_requested":
            # Prioriza review sobre tareas nuevas
            claim_review(msg["task_id"], msg["pr_url"])

    elif role == "project_owner":
        if msg["type"] == "qa_approved":
            # Evalua criterios de aceptacion
            evaluate_ticket(msg["task_id"], "po_acceptance")

        elif msg["type"] == "task_blocked":
            # Analiza blocker y re-prioriza si necesario
            analyze_blocker(msg["task_id"], msg["payload"])

    elif role == "tech_lead":
        if msg["type"] == "bug_found":
            # Prioriza fix del bug
            create_bugfix_task(msg["payload"])

        elif msg["type"] == "help_needed":
            # Ofrece guidance via MSG
            send_guidance(msg["from_machine"], msg["payload"])

    elif role == "developer":
        if msg["type"] == "contract_published":
            # Carga contrato para implementacion
            load_contract(msg["payload"]["contract_id"])
```

```
        elif msg["type"] == "qa_rejected":
            # Prioriza fix
            fix_qa_issues(msg["task_id"], msg["payload"])
```

---

6. Auto-Evaluacion de Tickets (Detalle)

6.1 Quality Gates automaticos

```
class TicketEvaluator:
    """Evalua si un ticket cumple todos los quality gates."""

    def evaluate_pre_release(self, task_id: str, repo_path: str) -> Eva
        """Evaluacion antes de hacer release_task."""
        checks = []

        # 1. Tests
        test_result = run_command(f"cd {repo_path} && pytest -v --tb=sh
        checks.append(Check("tests_pass", test_result.exit_code == 0, t

        # 2. Coverage
        cov_result = run_command(f"cd {repo_path} && pytest --cov --cov
        coverage = parse_coverage(cov_result.output)
        checks.append(Check("coverage_98", coverage >= 98, f"{coverage}

        # 3. Lint
        lint_result = run_command(f"cd {repo_path} && ruff check .")
        checks.append(Check("lint_clean", lint_result.exit_code == 0, l

        # 4. File size
        large_files = find_files_over_1000_lines(repo_path)
        checks.append(Check("no_large_files", len(large_files) == 0, la

        # 5. Function size
        large_funcs = find_functions_over_50_lines(repo_path)
        checks.append(Check("no_large_functions", len(large_funcs) == 0

        # 6. Type hints
        missing_hints = find_missing_type_hints(repo_path)
        checks.append(Check("type_hints", len(missing_hints) == 0, miss

        # 7. Commit format
        last_commit = get_last_commit_message(repo_path)
        valid_format = re.match(r'^(feat|fix|refactor|docs|test|chore)\
        checks.append(Check("commit_format", bool(valid_format), last_c

        # 8. Branch pushed
        is_pushed = check_branch_pushed(repo_path)
        checks.append(Check("branch_pushed", is_pushed))

        # 9. No secrets
        secrets = scan_for_secrets(repo_path)
```

```
        checks.append(Check("no_secrets", len(secrets) == 0, secrets))

        return EvalResult(
            task_id=task_id,
            evaluation_type="pre_release",
            passed=all(c.passed for c in checks),
            checks=checks,
            summary=generate_summary(checks)
        )

    def evaluate_qa_review(self, task_id: str, repo_path: str, pr_url:
        """Evaluacion QA de otra maquina."""
        checks = []

        # 1. Obtener archivos modificados en el PR
        changed_files = get_pr_changed_files(pr_url)

        # 2. Para cada archivo, verificar que tenga test correspondient
        for f in changed_files:
            test_file = find_test_for_file(f, repo_path)
            checks.append(Check(f"test_exists_{f}", test_file is not No

        # 3. Verificar test patterns
        test_files = [find_test_for_file(f, repo_path) for f in changed
        for tf in filter(None, test_files):
            has_happy = has_test_pattern(tf, "test_.*exitoso|test_.*suc
            has_error = has_test_pattern(tf, "test_.*error|test_.*falla
            checks.append(Check(f"happy_path_{tf}", has_happy))
            checks.append(Check(f"error_case_{tf}", has_error))

        # 4. No hay datos hardcodeados
        hardcoded = find_hardcoded_data_in_tests(repo_path)
        checks.append(Check("no_hardcoded_data", len(hardcoded) == 0, h

        # 5. Coverage del PR
        pr_coverage = get_pr_coverage(pr_url)
        checks.append(Check("pr_coverage_98", pr_coverage >= 98, f"{pr_

        return EvalResult(
            task_id=task_id,
            evaluation_type="qa_review",
            passed=all(c.passed for c in checks),
            checks=checks
        )

    def evaluate_po_acceptance(self, task_id: str) -> EvalResult:
        """Evaluacion de criterios de aceptacion por PO."""
        checks = []

        # 1. Obtener criterios de aceptacion del issue
        issue = get_issue_details(task_id)
        criteria = parse_acceptance_criteria(issue["body"])
```

```
        # 2. Para cada criterio, verificar que haya un test
        for criterion in criteria:
            test_exists = search_test_for_criterion(criterion)
            checks.append(Check(f"criterion_{criterion[:30]}", test_exi

        # 3. Verificar que no hay regresiones
        regression_result = run_full_test_suite()
        checks.append(Check("no_regressions", regression_result.exit_co

        # 4. Si es epica completa, verificar todos los criterios
        if is_last_story_in_epic(task_id):
            epic_criteria = get_epic_acceptance_criteria(task_id)
            for ec in epic_criteria:
                checks.append(Check(f"epic_{ec[:30]}", verify_epic_crit

        return EvalResult(
            task_id=task_id,
            evaluation_type="po_acceptance",
            passed=all(c.passed for c in checks),
            checks=checks
        )
```

6.2 Flujo de auto-evaluacion continua

```
Cada vez que una maquina completa una tarea:

1. SELF-EVAL (la misma maquina):
   covacha_evaluate_ticket(task, "pre_release")
   → Si FAIL: fix automatico + re-eval (max 3 intentos)
   → Si PASS: release + MSG "ready for QA"

2. QA-EVAL (mac-3 automaticamente):
   Recibe MSG → covacha_evaluate_ticket(task, "qa_review")
   → Si FAIL: MSG "qa_rejected" con detalles → maquina original fix
   → Si PASS: MSG "qa_approved"

3. PO-EVAL (mac-4 automaticamente):
   Recibe MSG → covacha_evaluate_ticket(task, "po_acceptance")
   → Si FAIL: MSG "po_rejected" con criterios faltantes
   → Si PASS: accept_ticket() → cierra issue → actualiza epica

Resultado: CERO intervencion humana en el ciclo completo.
```

---

7. Instalacion del MCP Server

7.1 Como MCP server en Claude Code (por maquina)

```
# Opcion 1: Desde npm (cuando este publicado)
claude mcp add covacha -- npx covacha-mcp-server

# Opcion 2: Desde path local (desarrollo)
```

```
claude mcp add covacha -- node /Users/casp/sandboxes/superpago/covacha-

# Opcion 3: En settings.json del proyecto
# .claude/settings.json
{
  "mcpServers": {
    "covacha": {
      "command": "node",
      "args": ["/path/to/covacha-mcp-server/dist/index.js"],
      "env": {
        "AWS_REGION": "us-east-1",
        "DYNAMO_TABLE": "covacha-agent-memory",
        "MACHINE_ID": "mac-1",
        "TEAM_ROLE": "tech_lead"
      }
    }
  }
}
```

7.2 Ambos MCP servers en paralelo

```
{
  "mcpServers": {
    "context-mode": {
      "command": "context-mode"
    },
    "covacha": {
      "command": "node",
      "args": ["/path/to/covacha-mcp-server/dist/index.js"],
      "env": {
        "MACHINE_ID": "mac-1",
        "TEAM_ROLE": "tech_lead"
      }
    }
  }
}
```

Claude Code ve 2 MCP servers:

- Context Mode: 6 tools (ctx_*) → ahorro de tokens + session continuity
- Covacha MCP: 14 tools (covacha_*) → coordinacion + roles + evaluacion

---

8. Implementacion: Plan por Fases

Fase A: MCP Server basico (2-3 dias)

- Setup proyecto TypeScript con MCP SDK
- Implementar 4 tools basicos: bootstrap, claim, release, team_status
- Conectar con DynamoDB via AWS SDK v3
- Tests con vitest

Fase B: Comunicacion (2 dias)

- Tools: send_message, read_messages
- Message processing por rol
- Tests de flujo completo

Fase C: Dependencies + Contracts (2 dias)

- Tools: check_dependencies, publish_contract, get_contract
- Integracion con release_task (auto-publish contracts)

Fase D: Auto-evaluacion (3 dias)

- Tool: evaluate_ticket (4 tipos de evaluacion)
- Quality gates automaticos
- Integracion con rol de tester y PO

Fase E: Roles + Agent Loop (3 dias)

- Tool: role_action
- Logica de rol por maquina
- Agent Loop integrado con MCP tools

Fase F: Dispatcher (2 dias)

- Tool: dispatcher_suggest
- Priority scoring
- Load balancing

---

9. Ventajas de MCP Server propio vs Scripts Python

Aspecto	Scripts Python (actual)	MCP Server (propuesto)
Invocacion	`python scripts/...` (manual)	Claude Code llama tool directamente
Contexto	Genera archivo CONTEXT.md	Retorna datos estructurados al LLM
Latencia	2-5 sec (subprocess)	<500ms (in-process via stdio)
Integracion	CLI separado del chat	Nativo en la conversacion
Hooks	No	Pre/Post tool hooks posibles
Sesion	Sin estado entre llamadas	Puede mantener cache en memoria
Discovery	Hay que saber que scripts existen	Claude ve tools automaticamente
Composicion	Scripts independientes	Tools se componen naturalmente